from start to finish, have also missed covering this change. Let's visualize this in an example build pipeline, as illustrated in figure 2.2.

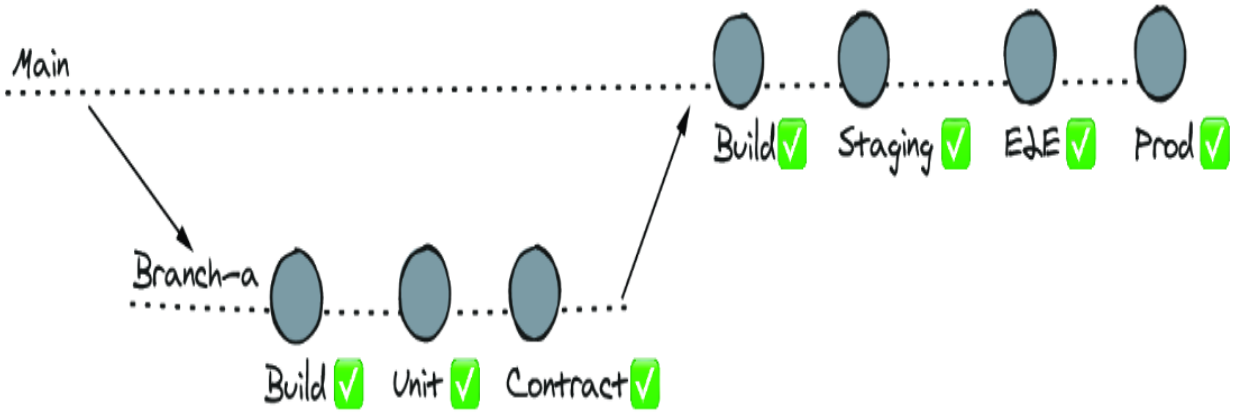


Figure 2.2 Build pipeline with consumer contract tests added, but no provider verification. This can lead to silent errors being deployed to production.

The build pipeline didn't throw any errors because no additional contract verification had been added. The absence of contract tests for the data providers led to a gap in knowledge, preventing teams from grasping the complete functionality and significance of contract testing. This situation also highlighted a lack of comprehensive understanding among the teams regarding the concept of contract testing and its role within the broader testing processes. Developers working on the GraphQL codebase wrote consumer contract tests just because it was being done on the other features.

If provider tests were in place in this project, breaking changes or contract errors, such as differences in status codes, data structure, or data types, could be detected and stop the build pipeline from progressing, as illustrated in figure 2.3, leading to errors being caught at an earlier stage. For example, if the data provider changed to rename some of the fields they expose to the GraphQL consumer, then this change would be caught by the contract verification process. Even if there is a schema in place in verifying the GraphQL structure, a schema can't tell you how consumers use it and what fields aren't used, which we explain further in chapter 3.